

TOYWORLD – E-COMMERCE DATABASE MANAGEMENT SYSTEM

ORDER AND ORDER_ITEM MANAGEMENT SYSTEM

Introduction:

The Orders and Order_Item tables are used to manage customer orders in the ToyWorld E-commerce Database Management System. The Orders table stores basic information such as Order ID, Customer ID, Order Date, Order Status, and Total Amount. The Order_Item table stores the products included in each order, along with their quantity and unit price. These tables help to maintain order information accurately and make it easy to insert, modify, and retrieve customer order history.

1. Database Design

1.1 Order Table:

Attribute	Key Type
Order_ID	Primary Key
Customer_ID	Foreign Key
Order_Date	-
Order_Status	-
Total_Amount	-

1.2 Order_Item Table:

Attribute	Key Type
OrderItem_ID	Primary Key
Order_ID	Foreign Key
Product_ID	Foreign Key
Quantity	-

2. Table Relationship

Customer → Orders

- Customer.Customer_ID → Primary Key

- Orders.Customer_ID → Foreign Key
- Relationship: One-to-Many (1)
- One customer can place multiple orders.

Orders → Order_Item

- Orders.Order_ID → Primary Key
- Order_Item.Order_ID → Foreign Key
- Relationship: One-to-Many (1)
- One order can contain multiple order items.

Product → Order_Item

- Product.Product_ID → Primary Key
- Order_Item.Product_ID → Foreign Key
- Relationship: One-to-Many (1)
- One product can appear in multiple order items.

3. Table Creation

3.1 Create Order Table

```
SQL> CREATE TABLE Orders (
2     Order_ID NUMBER PRIMARY KEY,
3     Customer_ID NUMBER,
4     Order_Date DATE,
5     Order_Status VARCHAR2(20),
6     Total_Amount NUMBER(10,2),
7     CONSTRAINT fk_order_customer
8     FOREIGN KEY (Customer_ID)
9     REFERENCES Customer(ID)
10 );
```

Table created.

3.2 Create Order_Item Table

```
SQL> CREATE TABLE Order_Item (  
 2     OrderItem_ID NUMBER PRIMARY KEY,  
 3     Order_ID NUMBER,  
 4     Product_ID NUMBER,  
 5     Quantity NUMBER,  
 6     Unit_Price NUMBER(10,2),  
 7     CONSTRAINT fk_orderitem_order  
 8         FOREIGN KEY (Order_ID)  
 9         REFERENCES Orders(Order_ID),  
10     CONSTRAINT fk_orderitem_product  
11         FOREIGN KEY (Product_ID)  
12         REFERENCES Product(Product_ID)  
13 );
```

Table created.

4. Order Data Insertion:

```
SQL> INSERT INTO Orders VALUES
  2  (1001, 1, TO_DATE('20-09-2026','DD-MM-YYYY'), 'Confirmed', 1200);

1 row created.

SQL>
SQL> INSERT INTO Orders VALUES
  2  (1002, 2, TO_DATE('21-09-2026','DD-MM-YYYY'), 'Shipped', 850);

1 row created.

SQL>
SQL> INSERT INTO Orders VALUES
  2  (1003, 3, TO_DATE('22-09-2026','DD-MM-YYYY'), 'Delivered', 1500);

1 row created.

SQL>
SQL> INSERT INTO Orders VALUES
  2  (1004, 4, TO_DATE('23-09-2026','DD-MM-YYYY'), 'Pending', 650);

1 row created.

SQL>
SQL> INSERT INTO Orders VALUES
  2  (1005, 5, TO_DATE('24-09-2026','DD-MM-YYYY'), 'Confirmed', 950);

1 row created.

SQL>
SQL> INSERT INTO Orders VALUES
  2  (1006, 1, TO_DATE('25-09-2026','DD-MM-YYYY'), 'Shipped', 1100);

1 row created.

SQL>
SQL> INSERT INTO Orders VALUES
  2  (1007, 2, TO_DATE('26-09-2026','DD-MM-YYYY'), 'Delivered', 1800);

1 row created.
```

5. Order_Item Data Insertion:

```
SQL> INSERT INTO Order_Item VALUES
  2 (1, 1001, 101, 2, 300);

1 row created.

SQL>
SQL> INSERT INTO Order_Item VALUES
  2 (2, 1002, 103, 1, 850);

1 row created.

SQL>
SQL> INSERT INTO Order_Item VALUES
  2 (3, 1003, 104, 1, 1500);

1 row created.

SQL>
SQL> INSERT INTO Order_Item VALUES
  2 (4, 1004, 105, 1, 650);

1 row created.

SQL>
SQL> INSERT INTO Order_Item VALUES
  2 (5, 1005, 108, 1, 950);

1 row created.

SQL>
SQL> INSERT INTO Order_Item VALUES
  2 (6, 1006, 109, 2, 550);

1 row created.

SQL>
SQL> INSERT INTO Order_Item VALUES
  2 (7, 1007, 101, 3, 600);

1 row created.
```

6. Order Records:

```
SQL> SELECT * FROM Orders;
```

ORDER_ID	CUSTOMER_ID	ORDER_DAT	ORDER_STATUS	TOTAL_AMOUNT
1001	1	20-SEP-26	Confirmed	1200
1002	2	21-SEP-26	Shipped	850
1003	3	22-SEP-26	Delivered	1500
1004	4	23-SEP-26	Pending	650
1005	5	24-SEP-26	Confirmed	950
1006	1	25-SEP-26	Shipped	1100
1007	2	26-SEP-26	Delivered	1800
1008	3	27-SEP-26	Confirmed	750

```
8 rows selected.
```

7. Order_Item Records:

```
SQL> SELECT * FROM Order_Item;
```

ORDERITEM_ID	ORDER_ID	PRODUCT_ID	QUANTITY	UNIT_PRICE
1	1001	101	2	300
2	1002	103	1	850
3	1003	104	1	1500
4	1004	105	1	650
5	1005	108	1	950
6	1006	109	2	550
7	1007	101	3	600
8	1008	103	1	750

```
8 rows selected.
```

8. Order Modification Operations

8.1 Modify Order Amount :

```
SQL> UPDATE Orders
2 SET Total_Amount = 1350
3 WHERE Order_ID = 1001;
```

```
1 row updated.
```

8.2 Modify Order Date :

```
SQL> UPDATE Orders
  2 SET Order_Date = TO_DATE('28-09-2026','DD-MM-YYYY')
  3 WHERE Order_ID = 1002;

1 row updated.
```

9. Order_Item Modification

```
SQL> UPDATE Order_Item
  2 SET Quantity = 4
  3 WHERE OrderItem_ID = 2;

1 row updated.

SQL> UPDATE Order_Item
  2 SET Unit_Price = 799.00
  3 WHERE OrderItem_ID = 5;

1 row updated.
```

10. Customer Order History Report:

```
SQL> SELECT o.Order_ID,
  2      o.Customer_ID,
  3      o.Order_Date,
  4      o.Order_Status,
  5      o.Total_Amount,
  6      oi.Product_ID,
  7      oi.Quantity,
  8      oi.Unit_Price
  9 FROM Orders o
 10 JOIN Order_Item oi
 11 ON o.Order_ID = oi.Order_ID
 12 ORDER BY o.Order_ID;
```

ORDER_ID	CUSTOMER_ID	ORDER_DATE	ORDER_STATUS	TOTAL_AMOUNT	PRODUCT_ID	QUANTITY	UNIT_PRICE
1001	1	20-SEP-26	Confirmed	1350.00	101	2	300.00
1002	2	28-SEP-26	Shipped	850.00	103	4	850.00
1003	3	22-SEP-26	Delivered	1500.00	104	1	1500.00
1004	4	23-SEP-26	Pending	650.00	105	1	650.00
1005	5	24-SEP-26	Confirmed	950.00	108	1	799.00
1006	1	25-SEP-26	Shipped	1100.00	109	2	550.00
1007	2	26-SEP-26	Delivered	1800.00	101	3	600.00
1008	3	27-SEP-26	Confirmed	750.00	103	1	750.00

8 rows selected.

11. Customer-Wise Order Summary:

```

SQL> SELECT Customer_ID,
2         COUNT(Order_ID) AS Total_Orders,
3         SUM(Total_Amount) AS Total_Amount
4 FROM Orders
5 GROUP BY Customer_ID
6 ORDER BY Customer_ID;

```

CUSTOMER_ID	TOTAL_ORDERS	TOTAL_AMOUNT
-----	-----	-----
1	2	2450.00
2	2	2650.00
3	2	2250.00
4	1	650.00
5	1	950.00

Conclusion:

The Order Management System was successfully designed using the Orders and Order_Item tables. The tables store important order information such as customer details, order date, order status, total amount, product, quantity, and unit price. Primary and foreign key relationships were used to maintain data integrity between Customer, Orders, Product, and Order_Item tables. Order records were inserted and modified using SQL commands. Customer order history and customer-wise order summaries were also generated to manage and analyze order information effectively.